SQL as a Domain-Specific Language for Migration Microsimulation

Abstract

Migration microsimulation models are essential tools for understanding population dynamics and regional planning. However, traditional implementations require substantial programming effort, with each demographic process implemented as custom code that must be compiled or interpreted within a general-purpose language. This paper proposes leveraging SQL as a domain-specific language (DSL) for migration microsimulation. We demonstrate that by using SQL as both a specification language and an execution engine, we can create a system where migration models are defined as data (SQL queries) rather than code. This approach, implemented in the Talos migration microsimulation engine, enables researchers to specify and modify migration models without programming expertise while maintaining the performance benefits of a compiled execution environment. The architecture supports age-specific migration probabilities, area tracking, configurable destination selection, and comprehensive migration statistics—core requirements of modern migration modeling.

1. Introduction

Migration microsimulation has become increasingly important for understanding population dynamics, regional planning, and policy evaluation. These models simulate individual-level transitions—migration between geographic areas, aging, mortality, fertility—and aggregate these individual histories to produce population projections and migration flows. The complexity of these models has grown substantially; contemporary implementations incorporate numerous modules with complex interdependencies, requiring careful ordering of transitions to maintain behavioral realism.

Despite their analytical power, migration microsimulation systems face a persistent challenge: the gap between model specification and implementation. Migration researchers typically conceive models in terms of transition probabilities, age-specific mobility rates, destination choice, and state changes—concepts that map naturally to declarative specifications. However, most implementations require translating these specifications into imperative code, creating barriers to model development and modification. This is particularly problematic in

migration modeling where parameters frequently change based on new census data or policy scenarios.

This paper proposes a different approach: using SQL as a domain-specific language for migration microsimulation. We argue that SQL's declarative nature, set-based operations, and mature optimization make it an ideal vehicle for expressing demographic transitions. The Talos engine demonstrates this approach, providing a self-contained, cross-platform tool where migration models are defined as SQL statements in configuration files. This enables researchers to focus on model specification rather than programming, while leveraging decades of optimization in SQL engines.

2. Domain-Specific Languages and the Lisp-SQL Connection

2.1 The Nature of Domain-Specific Languages

A domain-specific language is a programming language designed to solve a finite class of problems. Where general-purpose languages like Python or C++ provide maximum flexibility, DSLs offer focused expressiveness within a particular domain. The history of computing offers numerous examples: TeX for typesetting, MATLAB for numerical computation, and SQL itself for database manipulation.

DSLs succeed when they capture the vocabulary and concepts of a domain directly. A demographer thinking about migration thinks in terms of "age groups," "destination areas," and "migration probabilities"—not "for loops," "arrays," or "function calls." A well-designed DSL allows the domain expert to work in their own conceptual language, with the system handling the translation to executable code.

2.2 Lisp's Enduring Contribution: Code as Data

Lisp introduced a revolutionary concept: the program and the data it manipulates share the same representation. This property, known as homoiconicity, enables metaprogramming—writing programs that write programs. Lisp macros allow developers to create custom syntax structures that match problem domains without modifying the underlying compiler.

What made Lisp revolutionary for DSLs was the ability to treat code as data. When you write a program in Lisp, the program itself is just a list—the same data structure that Lisp manipulates. This means you can write programs that write other programs. This is not a

theoretical curiosity; it has practical implications for how we build and use domain-specific languages.

2.3 SQL as Lisp's Logical Descendant

SQL follows this same pattern in a more practical way. When we write an SQL query, we're writing a specification that describes what we want, not how to do it. The SQL engine decides how to execute it. Our migration models are just SQL strings stored in a configuration file—they're data. But when the simulation runs, that data becomes executable code.

The connection between Lisp and SQL is more than historical parallel. Both Lisp and SQL are declarative, both operate on structured data, and both can be interpreted or compiled. More importantly, SQL represents a successful DSL that has been optimized for data manipulation over decades. Its grammar, while finite, captures a rich set of operations: selection, projection, join, aggregation, window functions, and recursive queries. Modern implementations extend this with procedural extensions that transform SQL into a hybrid DSL for data-intensive computation.

2.4 Why This Matters for Migration Modeling

The key insight for our purposes is this: when we use SQL for microsimulation, the model becomes data. Migration rules are written as SQL text in a configuration file. The SQL engine executes them. The researcher writes specifications, not programs. "Move 8% of 18-34 year olds each year" is written directly as SQL that reads like English.

This matters because:

- Anyone who can write SQL can create migration models
- Models can be changed without touching code
- The same model works for any population size
- Everything is transparent and auditable
- Models can be shared, reviewed, and modified by domain experts

3. The SQL Migration Microsimulation Architecture

3.1 Migration Models as SQL Statements

The core insight of this approach is that migration transitions can be expressed as SQL UPDATE statements. Consider a simple age-based migration model:

```
UPDATE population
SET area =
  CASE
    WHEN age BETWEEN 18 AND 34
      AND alive = true
      AND random() < 0.08
    THEN CAST(abs(random() % 5) + 1 AS INTEGER)
    ELSE area
  END
WHERE alive = true
```

This statement captures the essential logic: individuals aged 18-34 face an 8% probability of migrating to a random new area. The SQL engine handles the iteration, conditional evaluation, and state update. More complex transitions are equally expressible:

```
-- Track previous area for migration history
UPDATE population
SET previous_area = area
WHERE alive = true;

-- Apply age-specific migration probabilities
UPDATE population
SET area =
  CASE
    WHEN age < 18 AND alive = true AND random() < {migration_rates.child
_0_17}
      THEN CAST(abs(random() % {num_areas}) + 1 AS INTEGER)
    WHEN age >= 18 AND age < 35 AND alive = true AND random() < {migrati
on_rates.adult_18_34}
      THEN CAST(abs(random() % {num_areas}) + 1 AS INTEGER)
    WHEN age >= 35 AND age < 65 AND alive = true AND random() < {migrati
on_rates.adult_35_64}
      THEN CAST(abs(random() % {num_areas}) + 1 AS INTEGER)
    WHEN age >= 65 AND alive = true AND random() < {migration_rates.elde
rly_65_plus}
      THEN CAST(abs(random() % {num_areas}) + 1 AS INTEGER)
    ELSE area
  END
WHERE alive = true
```

This example demonstrates several key features:

- **Age-specific probabilities**: Different rates for children, young adults, middle-aged, and elderly
- **Parameterization**: Rates are referenced as placeholders (e.g., {migration_rates.adult_18_34}) that can be changed without modifying the SQL
- **Previous area tracking**: The previous_area column captures migration history
- **Random destination selection**: Individuals move to a random area from a configurable set

3.2 The SQL Engine as Migration Interpreter

This architecture leverages the SQL engine not merely as a storage layer but as a domain-specific interpreter for migration models. Each model definition is a data element (a string containing SQL) that the system executes. This mirrors the Lisp philosophy where code and

data share representation: the model specification is both human-readable documentation and executable program.

Modern SQL engines provide sophisticated optimization and execution capabilities. SQL statements can be parsed into abstract syntax trees, transformed, and optimized. The same SQL statement that works for 100 people works for 10 million people—the database optimizes the execution. The researcher doesn't need to think about loops, indexes, or parallelization.

3.3 Migration Parameterization and Model Configuration

The approach becomes practical through parameterization. Rather than hardcoding migration probabilities, models reference parameters:

```
WHEN age BETWEEN 18 AND 34 AND alive = true
  AND random() < {migration_rates.adult_18_34}
  THEN CAST(abs(random() % {num_areas}) + 1 AS INTEGER)
```

A preprocessing step substitutes these placeholders with values from a configuration file. This separates model structure from model parameters—a migration researcher can adjust age-specific mobility rates without modifying the SQL statement, and can create new models by composing existing patterns.

The configuration file (written in YAML) contains all model parameters:

```
models:
  - name: "migration"
    priority: 3
    enabled: true
    parameters:
      migration_rates:
        child_0_17: 0.02
        adult_18_34: 0.08
        adult_35_64: 0.03
        elderly_65_plus: 0.01
      num_areas: 5
      query: |
        UPDATE population SET previous_area = area WHERE alive = true;
        UPDATE population SET area =
          CASE
            WHEN age < 18 AND alive = true AND random() < {migration_rat
es.child_0_17}
              THEN CAST(abs(random() % {num_areas}) + 1 AS INTEGER)
            -- ... other age groups
            ELSE area
          END
        WHERE alive = true
```

This separation of model structure from parameters has significant advantages:

- **Domain experts can modify rates**: A migration researcher can adjust age-specific probabilities without touching code
- **Scenario analysis**: Multiple parameter sets can be tested by simply changing values in the config file
- **Reproducibility**: The configuration file provides a complete record of model parameters
- **Collaboration**: Modelers can share and modify configurations without needing programming expertise

3.4 Migration Module Ordering and Dependencies

Migration microsimulation systems require careful ordering of transitions. Migration models typically run after aging and mortality to ensure that:

- Individuals have been aged to the correct age for migration probability calculations
- Only alive individuals are eligible to migrate

- Migration occurs before other transitions that might depend on area

The SQL approach accommodates this through a priority-based execution system:

```
models:
  - name: "age_increment"
    priority: 1
    query: "UPDATE population SET age = age + 1 WHERE alive = true"
  - name: "mortality"
    priority: 2
    query: "UPDATE population SET alive = false WHERE ..."
  - name: "migration"
    priority: 3
    query: "UPDATE population SET area = ... WHERE alive = true"
```

This explicit ordering allows modelers to specify dependencies declaratively, ensuring consistent state transitions across modules. The execution engine sorts models by priority and runs them sequentially, with lower numbers executing first.

3.5 Area Tracking and Migration History

A critical requirement of migration microsimulation is tracking migration history. The SQL approach supports this through additional columns that capture previous locations and migration events:

```
-- Save current area as previous area before migration
UPDATE population
SET previous_area = area
WHERE alive = true;

-- Track number of migrations
UPDATE population
SET migration_count = migration_count + 1
WHERE alive = true AND previous_area != area;

-- Record migration history in a separate table
INSERT INTO migration_history (person_id, year, from_area, to_area)
SELECT person_id, {CURRENT_YEAR}, previous_area, area
FROM population
WHERE alive = true AND previous_area != area;
```

This enables rich migration analysis, including:

- Migration flow matrices between areas
- Repeat migration patterns
- Age-specific migration corridors
- Duration of residence analysis
- Cumulative migration histories for individuals

3.6 Statistics and Analysis

The SQL approach naturally extends to statistical analysis. Any SQL query can be defined as a statistic and executed during the simulation:

```
statistics:
  - name: "migration_stats"
    description: "Migration statistics"
    query: |
      SELECT
        COUNT(CASE WHEN previous_area != area THEN 1 END) as migrants,
        COUNT(*) as total,
        CAST(COUNT(CASE WHEN previous_area != area THEN 1 END) AS FLOAT)
/ COUNT(*) * 100 as migration_rate_pct
      FROM population
      WHERE alive = true

  - name: "migration_by_age"
    description: "Migration by age group"
    query: |
      SELECT
        CASE
          WHEN age < 18 THEN '0-17'
          WHEN age < 35 THEN '18-34'
          WHEN age < 65 THEN '35-64'
          ELSE '65+'
        END as age_group,
        COUNT(CASE WHEN previous_area != area THEN 1 END) as migrants,
        COUNT(*) as total,
        CAST(COUNT(CASE WHEN previous_area != area THEN 1 END) AS FLOAT)
/ COUNT(*) * 100 as migration_rate_pct
      FROM population
      WHERE alive = true
      GROUP BY age_group
```

Statistics are computed each iteration and displayed in the output, providing real-time feedback on model behavior.

4. Advantages and Implications

4.1 Accessibility and Model Transparency

The SQL approach dramatically lowers barriers to model development. Demographers and migration researchers familiar with SQL (or learnable in hours) can specify models directly, without requiring programming expertise. The specification remains executable and auditable

—the model is the code.

This is particularly valuable in migration modeling where:

- Age-specific migration rates can be easily updated
- New destination choice mechanisms can be added
- Regional attractiveness factors can be incorporated
- Migration corridors can be defined explicitly

The configuration file serves as both documentation and executable specification, reducing the gap between model conceptualization and implementation.

4.2 Performance

SQL engines are highly optimized for set-based operations. A single SQL UPDATE can process millions of individuals efficiently, using indexes, vectorized execution, and query optimization. Migration calculations are particularly amenable to set-based operations, as migration probabilities are typically applied uniformly to population subsets defined by age groups.

In the Talos implementation, a pure Go SQLite engine runs entirely in memory, eliminating I/O bottlenecks while maintaining full SQL expressiveness. The system can process populations of 100,000 individuals with annual migration updates in seconds on commodity hardware.

4.3 Model Reuse and Composition

SQL's relational algebra provides natural composition mechanisms. Migration models can reference derived tables, use subqueries, or join across data sources. This enables the ability to link data sets together—the true strength of microsimulation.

For migration modeling, this means:

- Destination choice can depend on regional characteristics stored in external tables
- Migration flows can be calibrated to external constraints
- Multiple migration types (internal, international) can be modeled together
- Household-level migration can be implemented using joins on household identifiers

4.4 Implementation Considerations

The approach is feasible with any SQL engine. The Talos implementation uses a pure Go

SQLite engine, eliminating external dependencies and enabling single-binary deployment. SQLite's in-memory capability supports fast iteration while maintaining full SQL expressiveness. The implementation is designed for:

- Cross-platform compatibility (Linux, Windows, macOS)
- Zero installation requirements
- Rapid prototyping of migration scenarios
- Reproducible research through fixed random seeds

The system is packaged as a single executable file with no external dependencies—researchers can download and run it without installing databases, compilers, or runtime environments.

5. Extending the Migration Model

5.1 Destination Choice

The simple random destination selection can be extended to incorporate destination attractiveness:

```
-- Weighted random selection based on area attractiveness
UPDATE population
SET area = (
  SELECT area
  FROM area_attributes
  WHERE random() < cumulative_attractiveness / total_attractiveness
  ORDER BY cumulative_attractiveness
  LIMIT 1
)
WHERE alive = true AND random() < {migration_probability}
```

5.2 Distance-Based Migration

Migration probability can be modeled as a function of distance:

```
-- Migration probability decreases with distance
UPDATE population
SET area = destination_area
WHERE alive = true AND random() <
  {base_probability} * exp(-{distance_decay} * distance(current_area, de
stination_area))
```

This requires defining a distance function between areas, which can be implemented as a SQL function or look-up table.

5.3 Household Migration

Family migration can be modeled using SQL joins:

```
-- Migrate entire households together
UPDATE population
SET area = h.new_area
FROM household_migration h
WHERE population.household_id = h.household_id
  AND alive = true
```

This assumes a household_migration table has been populated with destination areas for households that choose to move.

5.4 Migration Calibration

Models can be calibrated to match aggregate migration flows:

```
-- Apply IPF-style calibration to migration probabilities
UPDATE population
SET migration_probability = migration_probability *
  (observed_flow / predicted_flow)
WHERE age_group = target_age_group
  AND origin_area = target_origin
```

This allows the model to be adjusted to match observed migration patterns at aggregate levels.

5.5 Regional Attractiveness

Migration destination choice can be modeled using regional characteristics:

```
-- Destination choice based on multiple factors
UPDATE population
SET area =
  CASE
    WHEN random() < {migration_probability} THEN
      (SELECT area
       FROM area_attributes
       WHERE random() <
         ({jobs_weight} * jobs_per_capita +
          {housing_weight} * housing_affordability +
          {amenities_weight} * amenity_score) /
         total_score
       LIMIT 1)
    ELSE area
  END
WHERE alive = true
```

6. Conclusion

The use of SQL as a domain-specific language for migration microsimulation draws on a rich heritage of DSL design. From Lisp's early insight that code and data can share representation to the modern SQL parser ecosystems, this approach offers a practical pathway to model-centric migration microsimulation.

The key contributions of this approach are:

1. **Accessibility**: Migration researchers can specify models directly using SQL, without programming expertise. The model specification is both human-readable documentation and executable code.

2. **Flexibility**: Models can be modified without recompilation. Changing migration rates or adding new behaviors is a matter of editing a configuration file and rerunning the simulation.

3. **Performance**: SQL engines are highly optimized for set-based operations, enabling efficient processing of large populations.

4. **Transparency**: The model specification is visible and auditable. There is no hidden code or compiled binaries—the model is the data.

5. **Composability**: SQL's relational algebra provides natural mechanisms for model composition, enabling complex behaviors to be built from simpler components.

By defining migration models as SQL statements, researchers can specify, modify, and compose demographic transitions declaratively, leveraging decades of optimization in SQL engines while maintaining the flexibility to express complex migration behaviors. The result is a system where models become data—auditable, parameterizable, and executable—embodying the Lisp principle of "code as data" in the context of migration simulation.

The Talos engine demonstrates the practicality of this approach, providing a self-contained, cross-platform tool for migration microsimulation that requires no programming expertise to use. As migration research increasingly requires rapid model iteration and policy scenario analysis, SQL-based microsimulation offers a compelling balance of flexibility, performance, and accessibility.

7. Future Work

Several directions for future development suggest themselves:

1. **Parallel execution**: Extending the system to process areas in parallel using Go's concurrency features.
2. **Visualization**: Integrating real-time visualization of migration flows and population distributions.
3. **Calibration tools**: Developing automated calibration procedures to match observed migration patterns.
4. **Household structures**: Supporting household-level migration decisions and household formation/dissolution.
5. **International migration**: Extending the area concept to include international migration.
6. **Temporal dynamics**: Modeling seasonal or multi-year migration patterns.
7. **Policy scenarios**: Enabling "what-if" analysis with different policy parameters.